



产品批次号产生规则 — 双模式方案

Seals MES System | 模式A：入库时自动生成 | 模式B：计划派发时预生成 | 系统管理可配置

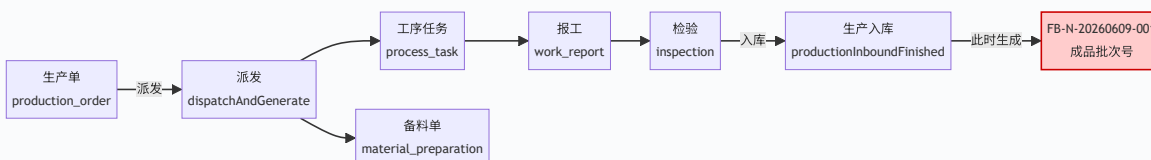
目录

- 现状分析
- 两种模式对比
- 模式A：入库时自动生成（现有）
- 模式B：计划派发时预生成（新增）
- 双模式协作流程
- 配置管理：产品批次号产生规则
- 数据库表设计
- 后端API设计
- 前端页面设计
- 核心代码改动点
- 批次号格式规范
- 边界场景与异常处理
- 实施路径

一、现状分析

1.1 现有批次号生成机制

当前系统中，成品批次号（FB 前缀）仅在**生产入库**环节由 `generateBatchNumber('FB', factoryCode, transaction)` 函数生成：



1.2 现有编号格式

批次类型	前缀	格式	示例
成品批次	FB	FB-{工厂编码}-{YYYYMMDD}-{序号}	FB-N-20260609-001
原料批次	MB	MB-{工厂编码}-{YYYYMMDD}-{序号}	MB-N-20260609-001
半成品批次	HB	HB-{工厂编码}-{YYYYMMDD}-{序号}	HB-N-20260609-001

1.3 序号管理

使用 `batch_number_sequence` 表管理每日递增序号，按 `prefix` 键（如 `FB-N-20260609-`）查询当前序号并自增，保证同一天同类型同工厂不重复。

1.4 痛点

- 无法提前追溯：批次号在入库时才产生，生产过程中无法用批次号关联计划→派发→报工→入库全链路
- 批次号与计划编号脱节：FB-N-20260609-001 与 M20260609001（生产计划编号）无逻辑关联
- 部分客户需求：某些客户/产品需要以计划编号为基准的批次号，便于从批次号直接反查生产计划

二、两种模式对比

● 模式A：入库时自动生成（现有）

- 触发时机：生产入库确认时
- 编号格式：FB-{FC}-{日期}-{序号}
- 优点：简单、无需预分配、入库数量确定后编号
- 缺点：无法提前追溯，与计划编号无关联
- 适用场景：通用产品、无追溯要求的常规入库

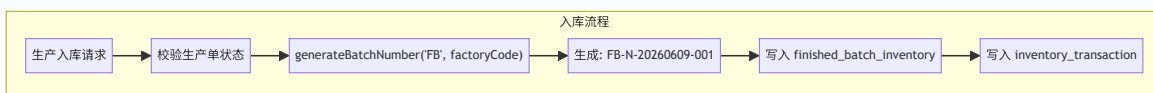
● 模式B：计划派发时预生成（新增）

- **触发时机**：生产计划拆分/派发时
- **编号格式**：FB-{计划编号后缀}
- **优点**：可提前追溯，批次号与计划强关联
- **缺点**：需预分配，入库数量可能变化
- **适用场景**：需追溯的定制产品、汽车件等

💡 **核心设计原则**：两种模式**并存**，由"产品批次号产生规则"配置决定每个产品使用哪种模式。未配置的产品默认使用模式A。

三、模式A：入库时自动生成（现有）

模式A保持现有逻辑不变，`generateBatchNumber('FB', factoryCode, transaction)` 在 `productionInboundFinished` 被调用时生成批次号：

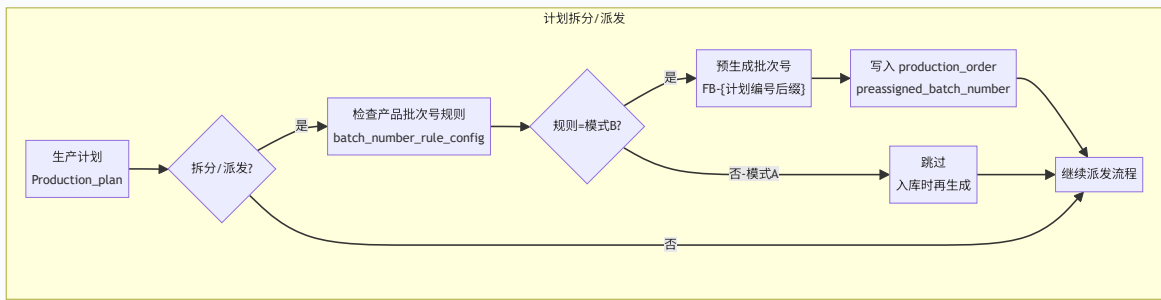


此模式下，成品批次号格式为 `FB-{工厂编码}-{YYYYMMDD}-{序号}`，序号由 `batch_number_sequence` 表管理。入库时生成，无需改动。

四、模式B：计划派发时预生成（新增）

4.1 触发时机

在生产计划被**拆分**或**派发**时，如果该产品配置了模式B，则预先生成成品批次号，并写入 `production_order` 的 `preassigned_batch_number` 字段。



4.2 批次号生成格式（模式B）

模式B的批次号以**生产计划编号**为基准：

元素	格式	示例	说明
前缀	FB-	FB-	成品批次统一前缀
计划编号	{production_number}	MN20260609001	生产计划编号
拆分序号	-S{nn}	-S01, -S02	同一计划拆分多单时的序号（不拆分则无）

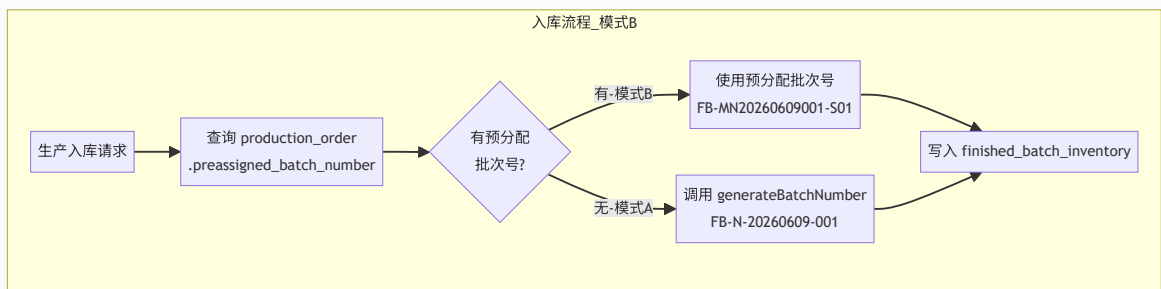
完整示例 FB-MN20260609001 或 FB-MN20260609001-S01

编号规则：

- 未拆分：FB-{production_number}（如 FB-MN20260609001）
- 拆分2单：FB-{production_number}-S01、FB-{production_number}-S02
- 拆分序号仅在 splitOrdersCore 时分配

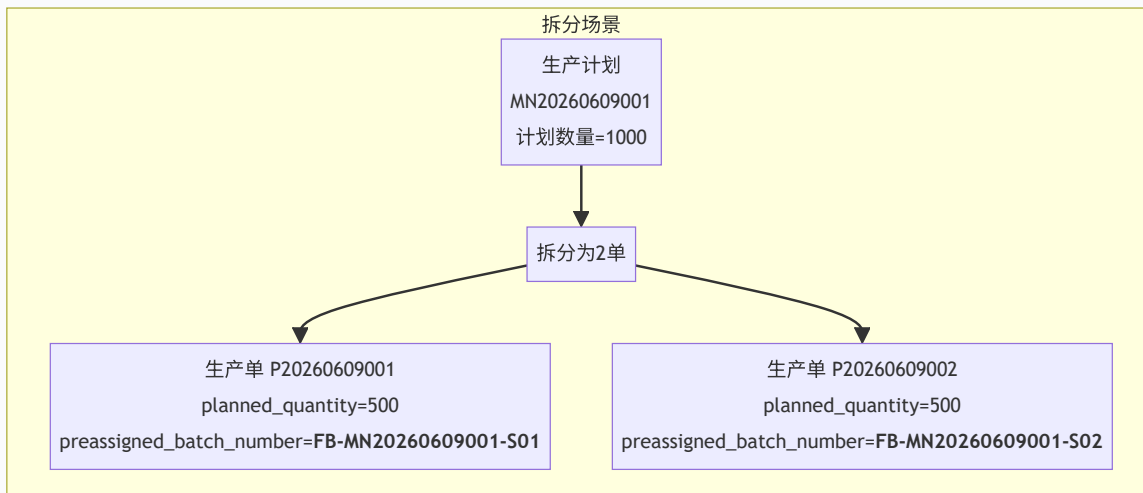
4.3 入库时使用预分配批次号

模式B下入库时，从 production_order.preassigned_batch_number 读取已分配的批次号，跳过自动生成：

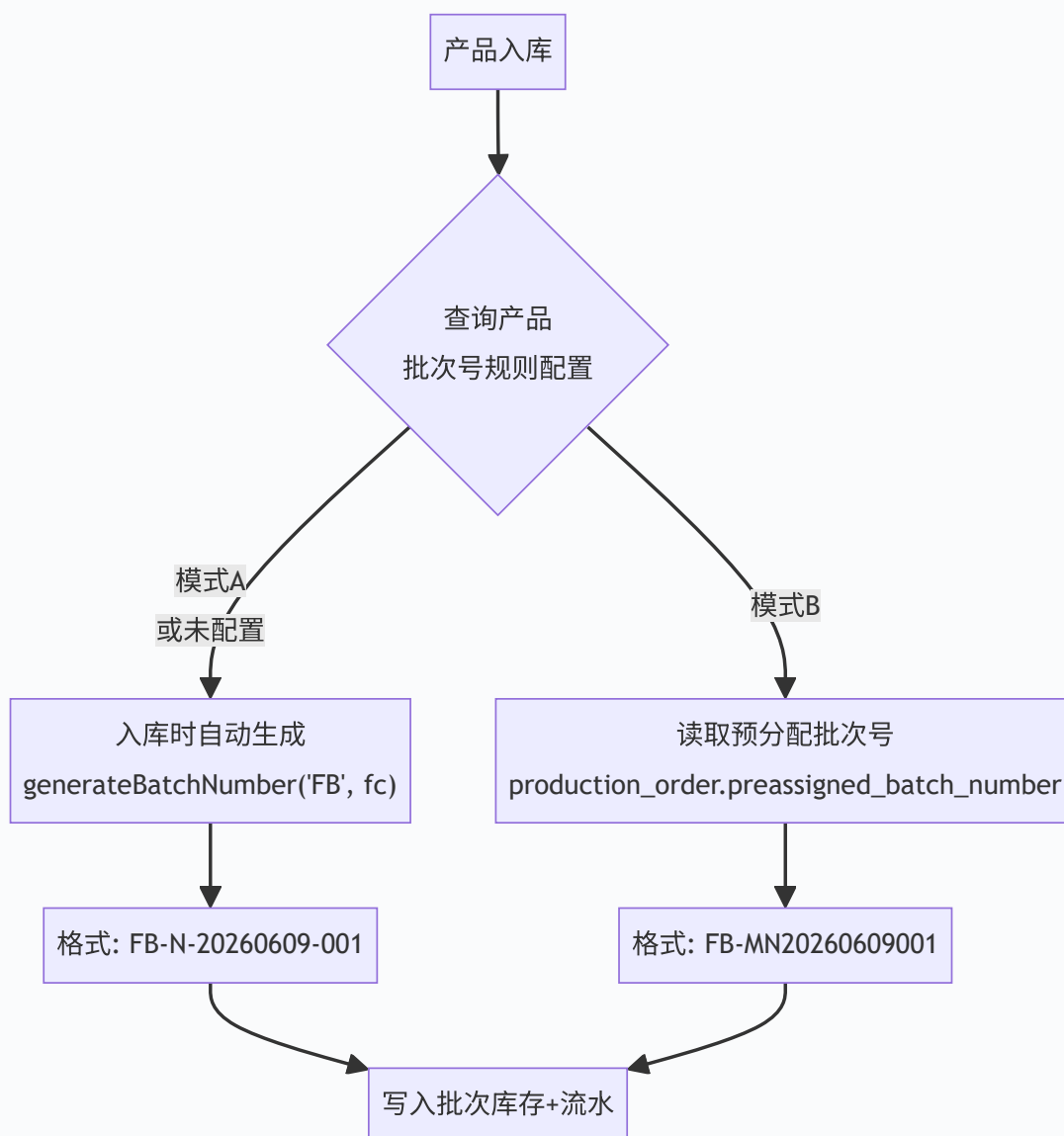


4.4 拆分场景的批次号分配

生产计划拆分时，原计划可能被拆成多个生产单，每个生产单都需要独立批次号：



五、双模式协作流程



⚠️ 一致性保证:

- 模式B的批次号在拆分/派发时已写入 `production_order`，入库时只读取，不会重新生成
- 模式B下如果 `preassigned_batch_number` 为空（异常情况），降级为模式A自动生成
- 撤回入库时，模式B的预分配批次号不清除（因为生产单仍存在），仅清除库存记录

六、配置管理：产品批次号产生规则

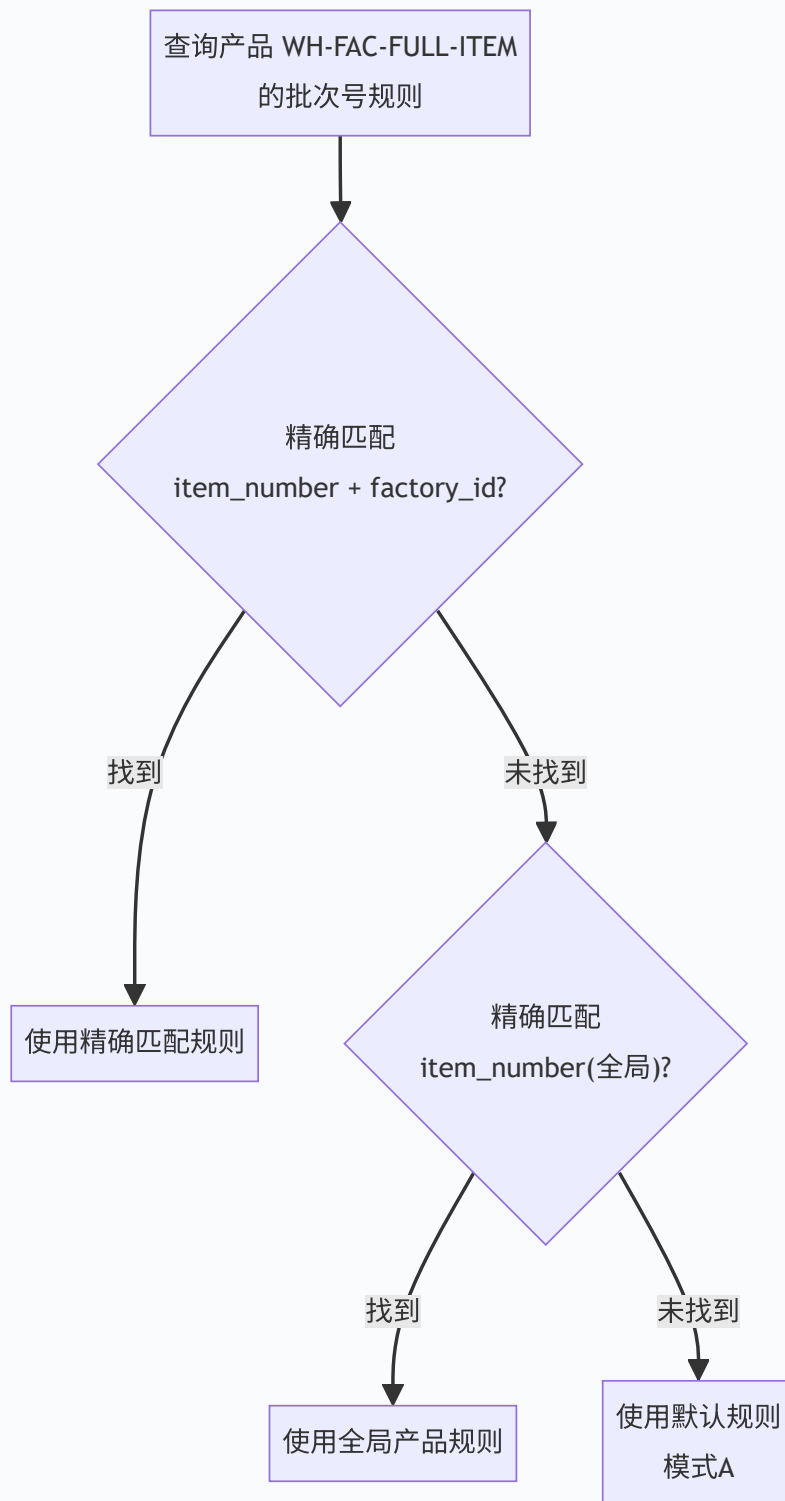
6.1 功能定位

在系统管理域下新增"产品批次号产生规则"管理页面，允许用户按产品维度配置批次号产生规则。

6.2 配置维度

配置项	说明	可选值	默认值
item_number	产品物料号	item_master 中任一成品	—
batch_rule_mode	批次号产生模式	A-入库时生成 B-计划派发时生成	A
batch_number_template	批次号模板（模式B）	FB-{plan_number}[-S{seq}]	FB-{plan_number}
factory_id	所属工厂（可选，空=全部工厂）	14/15/null	null（全部工厂）
is_active	是否启用	是/否	是

6.3 配置优先级



6.4 管理页面原型

页面位于 系统管理 → 产品批次号产生规则，标准 CRUD 页面：

列	类型	说明
产品编号	关联选择	从 item_master 中选择
产品名称	只读	自动带出

工厂	下拉	空=全部工厂
产生模式	单选	A-入库时生成 / B-计划派发时生成
批次号模板	文本	仅模式B时编辑，如 FB-{plan_number}
是否启用	开关	是/否
操作	—	编辑 / 禁用 / 删除

七、数据库表设计

7.1 新增表：batch_number_rule_config

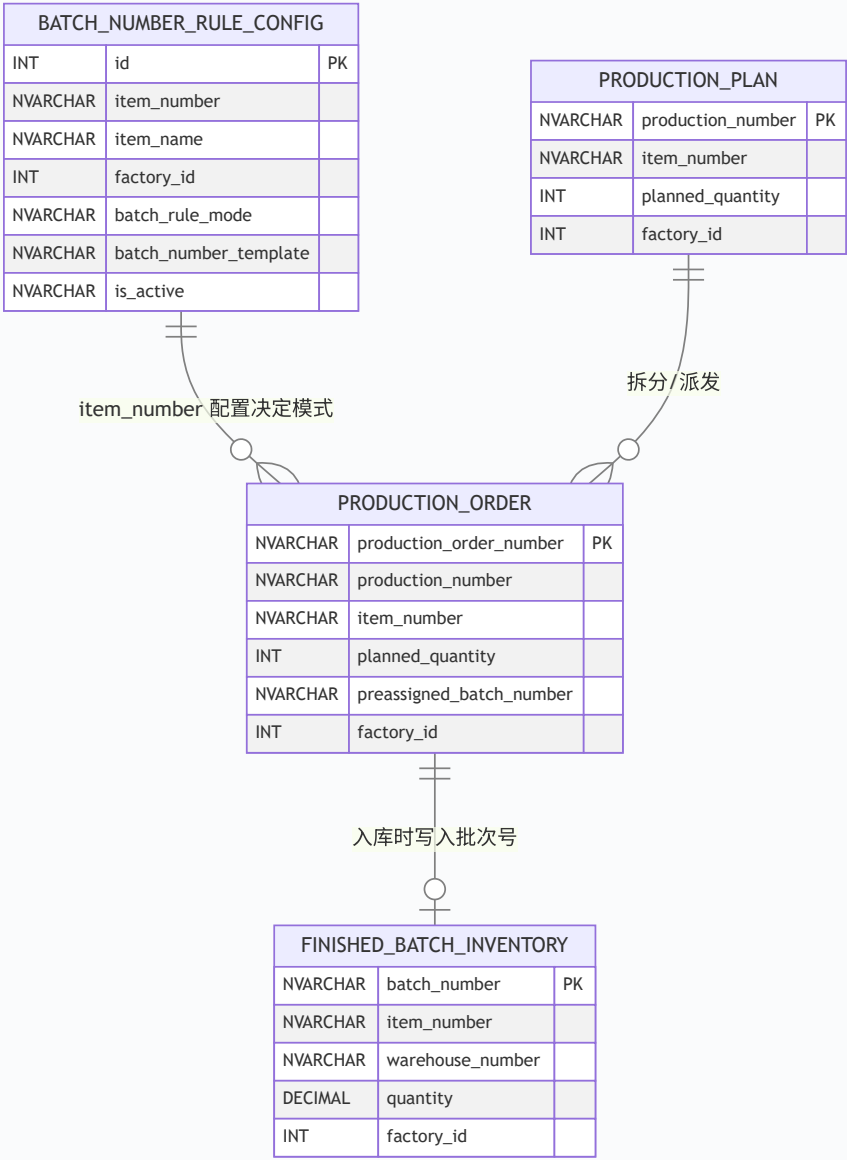
列名	类型	约束	说明
id	INT IDENTITY(1,1)	PRIMARY KEY	自增主键
item_number	NVARCHAR(100)	NOT NULL	产品物料号
item_name	NVARCHAR(200)	DEFAULT ''	产品名称（冗余）
factory_id	INT	NULL	所属工厂，NULL=全部工厂
batch_rule_mode	NVARCHAR(10)	NOT NULL DEFAULT 'A'	A=入库时生成, B=计划派发时生成
batch_number_template	NVARCHAR(200)	DEFAULT 'FB-{plan_number}'	模式B的批次号模板
is_active	NVARCHAR(10)	DEFAULT N'是'	是否启用
creation_date	DATETIME	DEFAULT GETDATE()	创建时间
creation_man	NVARCHAR(50)	DEFAULT ''	创建人
last_updated_at	DATETIME	DEFAULT GETDATE()	最后更新时间
last_updater	NVARCHAR(50)	DEFAULT ''	最后更新人

唯一约束： `UNIQUE (item_number, factory_id)` — 同一产品同一工厂只能有一条规则。
`factory_id=NULL` 表示全局规则。

7.2 修改表：production_order

新增列	类型	说明
<code>preassigned_batch_number</code>	<code>NVARCHAR(50) DEFAULT ''</code>	模式B下预分配的批次号，模式A下为空

7.3 ER 图



八、后端API设计

8.1 批次号规则配置 API

挂载在系统管理域: `/api/v1/system/batch-number-rules`

方法	路径	说明
GET	<code>/</code>	列表查询 (支持 item_number/工厂/模式筛选)
GET	<code>/:id</code>	详情
POST	<code>/</code>	创建规则
PUT	<code>/:id</code>	更新规则
DELETE	<code>/:id</code>	删除规则
GET	<code>/lookup?item_number=xxx&factory_id=14</code>	查询指定产品的生效规则 (含优先级解析)

8.2 核心服务函数变更

文件	函数	变更说明
<code>inventory.service.ts</code>	<code>generateBatchNumber()</code>	保持不变, 模式A仍用此函数
<code>inventory.service.ts</code>	<code>resolveBatchNumber()</code> 新增	统一入口: 查规则→模式A自动生成/模式B读取预分配
<code>finishedGoods.ts</code>	<code>productionInboundFinished()</code>	将 <code>generateBatchNumber('FB')</code> 替换为 <code>resolveBatchNumber()</code>
<code>dispatch.ts</code>	<code>splitOrdersCore()</code>	拆分时为模式B的产品预分配批次号
<code>dispatch.ts</code>	<code>dispatchAndGenerateCore()</code>	派发时为模式B的产品预分配批次号

8.3 resolveBatchNumber 伪代码

```
async resolveBatchNumber(params: {
  itemNumber: string,
  factoryId: number | null,
  productionOrderNumber: string,
  factoryCode: string,
  transaction: any
}): Promise<string> {
```

```
// 1. 查询产品批次号规则（含优先级）
const rule = await lookupBatchRule(params.itemNumber, params.factoryId);

// 2. 模式B：优先使用预分配批次号
if (rule?.batch_rule_mode === 'B') {
  const [po]: any = await sequelize.query(
    `SELECT preassigned_batch_number FROM production_order
    WHERE production_order_number = :pon`,
    { replacements: { pon: params.productionOrderNumber }, transaction }
  );
  if (po.length > 0 && po[0].preassigned_batch_number) {
    return po[0].preassigned_batch_number; // 模式B
  }
  // 降级：预分配为空则自动生成
  console.warn(`[resolveBatchNumber] 模式B但预分配为空，降级为模式A`);
}

// 3. 模式A（默认）：入库时自动生成
return generateBatchNumber('FB', params.factoryCode, params.transaction);
}
```

九、前端页面设计

9.1 菜单位置

系统管理 → 产品批次号产生规则

9.2 列表页功能

- 搜索栏：产品编号/名称、工厂筛选、产生模式筛选
- 操作栏：新增、导出
- 数据表：产品编号、产品名称、工厂、产生模式（标签色区分）、批次号模板、启用状态、操作
- 操作列：编辑、禁用/启用、删除

9.3 新增/编辑对话框

字段	控件	验证
产品编号	物料选择弹窗	必填，唯一（+工厂）

产品名称	只读文本	自动带出
工厂	下拉（含"全部"选项）	选填
产生模式	Radio：A-入库时生成 / B-计划派发时生成	必填
批次号模板	文本（仅模式B时可编辑）	模式B必填
是否启用	Switch	默认开启

9.4 前端路由

`/system/batch-number-rules` — 按系统管理页面标准布局

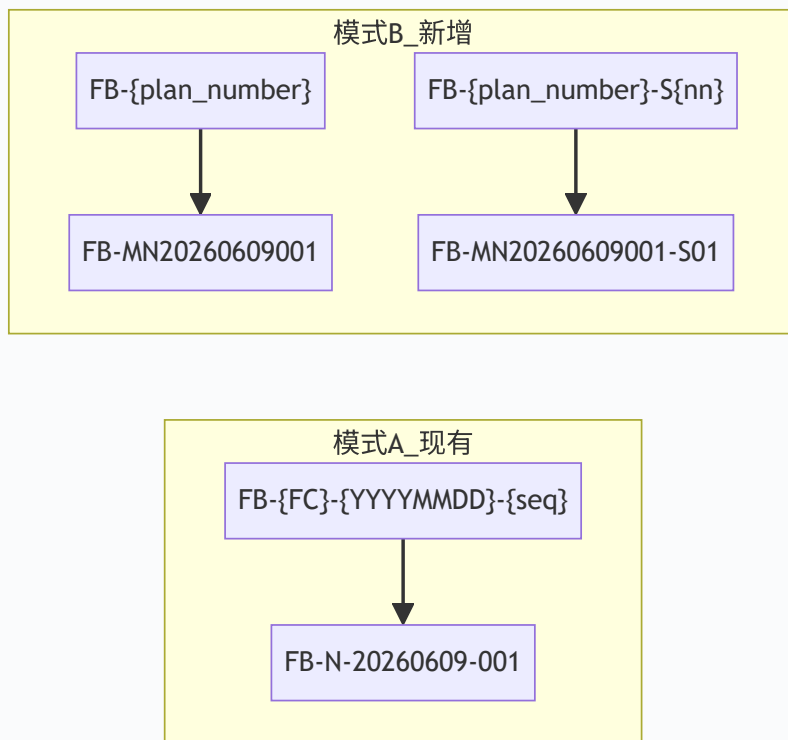
十、核心代码改动点

#	改动文件	改动内容	风险
1	<code>server/src/sql/migrations/lxx_batch_number_rule.ts</code>	新增迁移：创建 <code>batch_number_rule_config</code> 表 + <code>production_order</code> 增加 <code>preassigned_batch_number</code> 列	低
2	<code>server/src/services/inventory.service.ts</code>	新增 <code>resolveBatchNumber()</code> 和 <code>lookupBatchRule()</code>	低
3	<code>server/src/services/warehouse/finishedGoods.ts</code>	入库函数中 <code>generateBatchNumber('FB')</code> → <code>resolveBatchNumber()</code>	中
4	<code>server/src/services/orderDispatch/dispatch.ts</code>	<code>splitOrdersCore/dispatchAndGenerateCore</code> 中增加模式B预分配逻辑	中
5	<code>server/src/modules/system/batchNumberRule/</code>	新增 controller + routes + validator	低
6	<code>server/src/modules/system/index.ts</code>	注册新路由	低
7	<code>client/src/views/system/BatchNumberRule.vue</code>	新增管理页面	低
8	<code>client/src/router/</code>	添加前端路由	低
9	<code>client/src/api/system.ts</code>	添加 API 调用函数	低

⚠️ 兼容性保证:

- `generateBatchNumber()` 函数**不删除**，仅新增 `resolveBatchNumber()` 作为统一入口
- 所有现有调用点逐步迁移到 `resolveBatchNumber()`，未迁移前行为不变
- `batch_number_rule_config` 表初始为空，所有产品默认模式A，与现有行为完全一致

十一、批次号格式规范



模式B编号详解

场景	计划编号	生产单	预分配批次号
不拆分，直接派发	MN20260609001	P20260609001	FB-MN20260609001
拆分为2单	MN20260609002	P20260609002	FB-MN20260609002-S01
		P20260609003	FB-MN20260609002-S02
拆分为3单	MN20260609003	P20260609004	FB-MN20260609003-S01

P20260609005	FB-MN20260609003-S02
P20260609006	FB-MN20260609003-S03

长度约束： `finished_batch_inventory.batch_number` 为 NVARCHAR(50)。

- 模式A: FB-N-20260609-001 ≈ 19字符 ✓
 - 模式B无拆分: FB-MN20260609001 ≈ 19字符 ✓
 - 模式B有拆分: FB-MN20260609001-S01 ≈ 23字符 ✓
- 均在 50 字符限制内。

十二、边界场景与异常处理

场景	处理策略
模式B但 <code>preassigned_batch_number</code> 为空	降级为模式A自动生成，并记录警告日志
同一生产单多次入库（部分入库）	模式B：使用同一预分配批次号（与生产单 1:1） 模式A：每次入库生成新批次号
拆分后又撤回入库	预分配批次号不清除，仅回退库存记录
规则从模式B改为模式A	已预分配的批次号保留不影响，仅影响后续新单
规则从模式A改为模式B	仅影响新的拆分/派发，已有生产单不会追溯生成
删除规则	逻辑删除（ <code>is_active=否</code> ），不影响已使用批次号的数据
退货入库	退货入库不生成新批次号，使用原出库批次号（两种模式均如此）
异常出入库（盘盈入库）	不受规则影响，始终用 <code>generateBatchNumber('FB')</code> 自动生成

十三、实施路径



Phase	工期	说明
Phase 1	1天	迁移+服务函数, 风险低
Phase 2	2天	核心逻辑改动, 需充分测试
Phase 3	1.5天	标准CRUD页面
Phase 4	1天	E2E+边界测试
合计	5.5天	